

SpecFSM: Field-Centric Reasoning and Test Validation for Protocol Specification Compliance

Anonymous Authors
Paper under double-blind review

Abstract

Protocol implementations are expected to follow natural-language standards, yet small semantic deviations in field handling, state-dependent checks, and error paths can silently undermine interoperability and security. Existing conformance techniques remain limited by costly manual formalization or weak links between normative text and code evidence. This paper presents SpecFSM, a framework that checks protocol implementations against standards by using protocol fields as the semantic bridge between natural-language rules and executable code. SpecFSM first searches variables and extracts field-centric rules from standards, maps each rule to implementation variables and contexts, and then uses a reasoning-and-test agent to distinguish consistent behavior, inconsistent behavior, and insufficient evidence. On real-world implementations of TLS, MQTT, and QUIC, SpecFSM generated 204 conformance-deviation reports. Of these reports, 126 have been confirmed and fixed by maintainers, roughly nine were false positives, and the remaining reports are awaiting maintainer response or final adjudication; two confirmed issues became previously unknown CVEs. Using GPT-5.4 pricing as the cost model, SpecFSM costs \$1.122 per submitted report, suggesting that field-constrained LLM reasoning can be practical when paired with code-context extraction and reproducible validation artifacts.

1 Introduction

Modern network services depend on implementations of protocol standards such as TLS [43], MQTT [38, 39], and QUIC [28, 29, 46]. These standards specify how endpoints authenticate peers, negotiate cryptographic parameters, reject malformed messages, preserve session state, and interoperate across independent implementations. Even when the standard is clear, an implementation can still diverge in subtle field-handling, state-checking, or error-path behavior. Such implementation deviations are not merely benign compatibility quirks: they may cause an implementation to accept a

forbidden field value, skip a required state-dependent check, reject a legal message, or expose different behavior under a build option that deployers rarely test. The consequences range from interoperability failures to downgrade opportunities and denial-of-service conditions.

Checking these deviations is difficult because the two sides of the problem are written in radically different languages. Standards are natural-language documents whose normative requirements are distributed across field tables, message descriptions, state-machine prose, compatibility notes, and exception clauses. RFC-style normative language provides useful cues [7, 30], but those cues do not by themselves identify executable code obligations. Implementations are engineering artifacts with macros, configuration flags, generated code, helper functions, callbacks, and optimized error paths. The most important evidence for a single rule may be split across a parser, a state transition, a validation helper, and a caller-imposed precondition. A checker must therefore answer a consistency question rather than a mere localization question: under the conditions described by a normative rule, does the implementation enforce the required behavior on the relevant code paths?

Existing approaches cover only parts of this question. Formal methods can prove strong properties once a formal model exists, but constructing such a model from an evolving RFC or industry standard remains expensive and requires expertise. Protocol fuzzing and conformance tests exercise implementations, but they often use test oracles that are written separately from individual normative requirements, making it difficult to tell which field-level rule a test validates or leaves unchecked. Static analysis can reason over code structure, yet it typically has no access to the semantics of the standard. Recent large language models provide a new opportunity: their ability to jointly interpret natural-language requirements and source code makes it possible to bridge standards and implementations without requiring a complete hand-written formal model. However, this opportunity is useful for conformance checking only when the model is given precise obligations and the code evidence needed to evaluate them.

This paper is based on a simple observation: although protocol standards are written as prose, many checkable requirements are ultimately anchored in fields. A standard may state that a field must be present only under a condition, that a length must match another field, that a value must be derived from a negotiated parameter, or that a malformed combination must trigger an alert. These rules can be represented as field-centric assertions of the form $\langle C, f, A \rangle$: under condition C , field f must satisfy action or constraint A . This representation is not a full formal semantics of the protocol. Instead, it is a practical intermediate layer for organizing evidence, aligning specifications to code, and making uncertainty explicit when the current evidence is not enough.

We present SpecFSM, a framework for checking whether protocol implementations satisfy field-level requirements in natural-language standards. SpecFSM first constructs a field space from message structures, extensions, and field definitions, then extracts field-centric rules from normative prose. It next maps each rule to code by combining LLM-based semantic ranking, identifier normalization, Tree-sitter-based C/C++ syntax evidence, and deterministic filtering. Rather than asking a model for a one-shot verdict, SpecFSM performs iterative consistency reasoning: the model must report supporting evidence, missing evidence, and one of three outcomes: consistent, inconsistent, or insufficient evidence. When the outcome is suspicious or under-supported, a test-validation agent attempts to construct an executable counterexample that confirms or refutes the static hypothesis.

The goal is not to make the LLM itself the checker. Instead, SpecFSM turns the model’s language-and-code understanding into an evidence-producing workflow: field-centric rules define what must be checked, syntax-aware retrieval collects the relevant implementation context, consistency reasoning records supporting and missing evidence, and validation tests turn high-risk static judgments into observable behavior when possible. The result is a framework that can process natural-language standards while still producing reproducible conformance claims.

We evaluate SpecFSM on real-world implementations covering TLS, MQTT, and QUIC. SpecFSM generated 204 conformance-deviation reports. Maintainers have confirmed and fixed 126 of them, roughly nine reports were false positives, and 69 are still awaiting response or final adjudication. Two confirmed reports correspond to previously unknown CVEs. The analysis is also inexpensive relative to manual audit: using GPT-5.4 pricing as the cost model, SpecFSM spends \$1.122 per submitted report on average. These results suggest that LLMs can support protocol conformance checking when their judgments are grounded in field-level rules, implementation context, and validation artifacts.

Our main contributions are as follows:

- We formulate protocol conformance checking as a field-centric specification-to-implementation consistency prob-

lem, using $\langle C, f, A \rangle$ rules to represent checkable obligations from natural-language standards.

- We design SpecFSM, an evidence-driven pipeline that extracts field-level rules, aligns them to implementation variables and code contexts, reasons over consistency with explicit missing-evidence states, and validates suspected inconsistencies with executable tests when possible.
- We provide an empirical study, ablations, and case studies over real-world protocol implementations, producing 204 submitted reports, 126 maintainer-confirmed fixes, roughly nine false positives, and two previously unknown CVEs.

2 Background and Motivation

2.1 Field-Level Conformance

Protocol conformance asks whether an implementation satisfies the behavioral requirements stated by a standard. For natural-language standards, this is a specification-to-code alignment problem: the checker must connect a normative requirement to the protocol object it constrains and to the implementation behavior that determines whether the requirement is satisfied. Incorrect alignment can produce a wrong conclusion, for example when a length rule is matched to an unrelated buffer or when a validation path is missed because it is enforced outside the parser.

This work focuses on *field-level semantic conformance*. Fields are a useful boundary because standards define them through message formats, parameter tables, and normative prose, while implementations eventually represent them through parsed values, state variables, or validation logic. The names may differ, but the field gives the checker a stable object to track. This scope is narrower than full protocol verification, but it covers many practical defects because parsing, validation, and error handling are usually organized around fields.

2.2 Normative Modalities

The strength of a conformance claim depends on normative modality. Requirements phrased with *MUST* or *MUST NOT* usually support direct violation reports. Requirements phrased with *SHOULD* are tracked but reported as lower-severity deviations, while *MAY* usually indicates optional behavior and is ignored as a primary check target. SpecFSM records modality as part of each rule and uses it to rank report severity. A *MUST NOT accept* rule with a reproducible accepting input is reported as a strong inconsistency. A *SHOULD reject* rule with similar evidence is reported as a *SHOULD*-level deviation rather than a low-confidence result. This distinction is important because protocol standards often encode both hard interoperability constraints and softer deployment guidance.

2.3 A Running Example

Consider a simplified QUIC transport rule: if an endpoint receives a transport parameter that violates a specified bound, it must terminate the connection with a protocol error. The standard may express this requirement across a field definition, a paragraph about transport parameters, and an error-handling section. The implementation may parse the parameter in one file, store it in a connection configuration structure, validate it in a helper, and trigger the error through a callback. A direct text-to-code query such as “does the code enforce the transport parameter bound?” is under-specified: the relevant variable names, call chain, and build configuration are all missing.

SpecFSM handles the example by first extracting a rule such as

```
C = parameter received,  
f = max_udp_payload_size,  
A = reject values below the lower bound.
```

It then searches for identifiers and code regions related to the field, including aliases such as `max_udp_payload`, `udp_payload_size`, or structure members embedded in parser state. If the first retrieved context only shows assignment but not validation, the reasoning agent emits *insufficient evidence* and requests callers or validation helpers. Only after the evidence covers the relevant path does the system report a consistency judgment. For a suspected deviation, the test agent attempts to construct a packet or configuration that exercises the suspicious path and observes whether the implementation accepts or rejects it.

3 Method

Figure 1 shows the SpecFSM pipeline. The system takes two inputs: a natural-language protocol standard and a project codebase. It then executes three analysis stages before producing outputs: (1) LLM-based rule extraction from the standard, (2) code-context extraction from the implementation, and (3) a reasoning-and-test agent that combines both streams of evidence. The output is not only a label, but a summary report, a bug report, and reproduction artifacts.

Standard-document analysis. The left-top input box captures what makes standards hard to analyze: natural-language requirements, variables and states, and boundary or exception logic. SpecFSM first searches for protocol variables and state-bearing objects, then extracts field-centric rules from the relevant text. This two-stage design avoids asking an LLM to directly turn arbitrary prose into bugs.

Code-context extraction. The left-bottom input box represents implementation evidence: C files, headers, structures, enums, macros, functions, and call graphs. SpecFSM maps each standard variable to implementation-level identifiers and

then extracts the code contexts needed to evaluate the corresponding rule. This produces the code evidence consumed by the reasoning agent.

Reasoning-and-test agent. The agent first performs consistency reasoning and may return *insufficient*, *consistent*, or *inconsistent*. Insufficient evidence triggers another code-context extraction round through the blue data flow edge. For inconsistent or unresolved cases, the verification-test component generates a test plan, builds and runs the test, and judges the observed behavior.

Reports and artifacts. SpecFSM produces three classes of outputs: a summary report for triage, a bug report for maintainers, and reproduction artifacts for validation. This output structure is central to the paper: the goal is not merely to ask an LLM whether a rule is violated, but to produce auditable evidence that maintainers can act on.

3.1 Design Principles

SpecFSM follows three design principles.

Constrain before asking the model. The system narrows the problem before each LLM call. Rule extraction is limited by the field space. Code reasoning is limited by ranked snippets. Test planning is limited by an explicit defect hypothesis. This structure is intended to preserve the model’s semantic flexibility while reducing opportunities for ungrounded claims.

Prefer evidence over verdicts. Every stage emits evidence, not just a label. The rule extractor cites source spans, the aligner records why a variable was selected, the reasoner cites code snippets, and the test agent records commands and logs. This evidence-first style is essential for top-tier review because a detected conformance issue is only convincing if readers can trace it from standard text to implementation behavior.

Treat uncertainty as progress. Many true bugs initially look like missing context, and many false positives initially look like missing checks. SpecFSM therefore treats uncertainty as a structured intermediate result. A report can remain unresolved with a precise reason, such as a missing macro definition or an unreachable callback. This prevents the framework from inflating its bug count by converting every unknown into an inconsistency.

3.2 End-to-End Walkthrough

This section illustrates the end-to-end behavior of SpecFSM using a simplified transport-parameter example. The purpose is to make the data transformations concrete before presenting the full implementation details.

3.2.1 From Standard Text to a Rule

Suppose a transport standard states that an endpoint receiving a parameter below a specified lower bound must treat the mes-

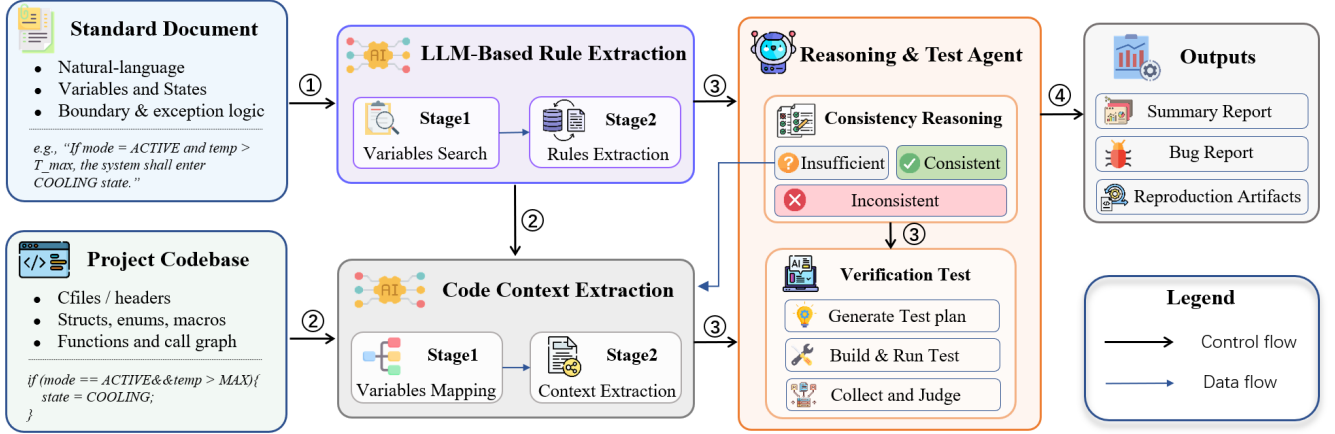


Figure 1: Overview of SpecFSM. The original system design has four modules: standard-document analysis, LLM-based rule extraction, code-context extraction, and a reasoning-and-test agent that emits summary reports, bug reports, and reproduction artifacts. Black arrows denote control flow and blue arrows denote data flow.

sage as a protocol error. The sentence is not enough by itself. The checker must know which field is constrained, which condition activates the rule, what behavior is required, and where the requirement appears in the document. SpecFSM first resolves the parameter mention against the field space:

C : parameter is present in peer transport parameters,
f : max_udp_payload_size,
A : reject values below 1200 with a protocol error,
M : MUST.

The rule retains the original sentence and section number. If another section defines the bound or error code, that section is attached as a dependency.

3.2.2 From Rule to Code Evidence

The aligner expands the field name into aliases such as max_udp_payload_size, max_payload, and udp_payload. Exact matches alone are insufficient: one file may define the transport-parameter identifier, another may parse the integer, and a third may validate the configured connection state. SpecFSM therefore retrieves a bundle containing the parser assignment, the validation helper, and the error return path. Each snippet is tagged with why it was selected: exact field hit, symbol scope, action-token overlap, or caller expansion.

3.2.3 From Evidence to a Judgment

In the first reasoning round, the model sees the assignment but not the caller that checks the helper’s return value. A one-shot checker may incorrectly report a missing rejection path. SpecFSM instead emits INSUFFICIENT with a typed request for callers of the validation helper. In the second round, the caller evidence shows that the error return is propagated to

connection close. The final label becomes CONSISTENT. In a different implementation, the second round might show that the lower-bound check is absent while nearby upper-bound checks are present, yielding a defect hypothesis.

3.2.4 From Hypothesis to Validation

For a suspected mismatch, the validation agent constructs a parameter block whose value violates the lower bound. It then chooses the closest available entry point: a unit-test parser if one exists, otherwise a small harness around the parser function, otherwise a command-line or integration test. The oracle is derived from the rule: the implementation should reject the input with the specified protocol error. The final report records the input, command line, build options, observed return value, and log excerpt. If the harness cannot reach the relevant path, the report remains unresolved rather than confirmed.

This walkthrough highlights the main difference between SpecFSM and direct LLM inspection. The model is never asked to solve the entire problem from a large prompt. It receives a constrained rule, a small evidence bundle, and an explicit choice to request more evidence when the bundle is incomplete.

4 Design Details

4.1 Problem Definition

Given a protocol standard *S* and an implementation *P*, SpecFSM seeks to identify whether *P* satisfies a set of normative field-level requirements in *S*. A requirement is represented as a rule $r = \langle C, f, A, M, \Pi \rangle$, where *C* is the applicability condition, *f* is the protocol field, *A* is the expected action or constraint, *M* is the normative modal-

ity such as *MUST* or *SHOULD*, and Π records source provenance. The checker returns a tuple $\langle y, E, U, T \rangle$: label $y \in \{\text{CONSISTENT}, \text{INCONSISTENT}, \text{INSUFFICIENT}\}$, evidence E , uncertainty notes U , and optional validation-test artifact T .

SpecFSM does not assume that the implementation uses names similar to the standard. It also does not require a pre-existing formal model. Instead, it builds a structured intermediate representation from the standard and aligns that representation to code using a combination of semantic and syntactic evidence.

4.2 Field-Space Construction

Directly extracting rules from open-ended prose is unstable because standards mix design motivation, compatibility advice, examples, and normative requirements. SpecFSM therefore starts by constructing a *field space*: a set of protocol objects that subsequent rules may reference. Field candidates are collected from message formats, extension structures, parameter tables, and explicit field-definition paragraphs. For each field, SpecFSM records its canonical name, aliases, parent message or structure, type, allowed values when available, and source locations.

Document preprocessing. SpecFSM ingests RFC-style text, markdown, and text extracted from PDFs. It segments each standard by section, preserves sentence offsets, and keeps layout-derived regions such as message diagrams and tables as separate evidence blocks. Field-space construction uses these regions before prose-only evidence: diagram labels and table columns usually identify protocol objects more reliably than later explanatory text. Ambiguous candidates are retained with their local context so that the rule extractor can resolve them against nearby normative language rather than forcing an early merge.

The field space serves two purposes. First, it gives the extractor a closed set of referents, reducing the chance that descriptive concepts such as “compatibility mode” are mistaken for checkable fields. Second, it preserves cross-document identity: the same field may appear as a full path, short name, structure member, or contextual pronoun. During rule extraction, all references must resolve to one or more field-space entries. Ambiguous references are kept as uncertainty rather than silently collapsed.

Field identity and validation. SpecFSM treats two mentions as the same field only when their parent context and semantic role are compatible, preventing generic names such as *length*, *type*, or *flags* from being merged across unrelated messages. Before rule extraction, it also validates the field space by marking entries with missing parent context, incompatible aliases, or only weak prose evidence as ambiguous or out of scope. This conservative policy favors precision over recall because an incorrect field merge can mislead all downstream alignment and reasoning.

Table 1: Rule schema used by SpecFSM.

Attribute	Description
Field	Constrained protocol field or parameter
Condition	Message type, state, phase, or field predicate
Action	Presence, value, derivation, relation, or rejection
Modality	MUST, MUST NOT, SHOULD, MAY, or equivalent phrase
Evidence	Source section, sentence span, and referenced text
Dependencies	Other fields or states needed to evaluate the rule

4.3 Field-Centric Rule Extraction

After field-space construction, SpecFSM extracts normative requirements as field-centric rules. The extraction prompt is constrained by the available field space and by a fixed schema. The model must identify the constrained field, condition, action, modality, source span, and any dependent fields. A deterministic checker then validates that the field appears in the field space, that the source span exists, and that the action belongs to a supported action family.

Prompt scope and validation. The extractor receives one standard section, the relevant field-space entries, and a conservative lexicon of normative keywords derived from RFC 2119-style language. It does not see the whole standard. This boundary keeps extraction local enough for reviewers to audit while still allowing cross-references to be resolved through field-space identifiers. Each model output is a typed record that is checked for schema validity, source-span existence, duplicate rules, field-space references, and action-family membership. Malformed records are retried with a compact repair prompt; if validation still fails, the section is marked as insufficient evidence rather than converted into an unverifiable rule.

The action families are intentionally close to implementation behavior: *assign*, *derive*, *select*, *require-present*, *require-absent*, *compare*, *range-check*, *state-update*, and *reject*. This design encourages a rule to map to observable code constructs such as assignments, conditionals, error returns, and state transitions. When a sentence contains multiple constraints, SpecFSM atomizes it into multiple rules so that each rule can be aligned and tested independently.

Rule normalization. Before alignment, rules are normalized to make equivalent requirements compare more consistently. Negated forms are rewritten into explicit actions, e.g., “MUST NOT accept x ” becomes an expected *reject* action under condition x . Cross-references are expanded when the referenced section contains a field-space entry. Multiple modalities in the same sentence are split into separate rules. The normalized rule remains linked to the original text so that reviewers can audit whether the transformation preserved the

intended meaning.

Unsupported rules. Some normative statements are intentionally not converted into field-level rules. Examples include broad performance guidance, implementation advice without observable behavior, and cryptographic assumptions that require a proof rather than code-level checking. SpecFSM records these statements as *out-of-scope normative text*. Reporting out-of-scope text is useful: it prevents silent loss of requirements and makes the method’s coverage explicit.

4.4 Syntax-Aware Rule-to-Code Alignment

The alignment stage maps a rule r to implementation variables and code contexts. It proceeds in three steps.

File ranking. SpecFSM uses an LLM-based ranker to score candidate C/C++ files by their semantic relationship to the rule. The ranker sees the rule, field aliases, action family, and file-level summaries. It cannot introduce files outside the provided implementation. The top-ranked files are deduplicated and passed to localization.

Variable localization. For the constrained field f , SpecFSM generates aliases by splitting field tokens, normalizing case conventions, expanding common abbreviations, and including parent-structure names when available. It then constructs a symbol index using Tree-sitter [9] with C/C++ grammars and compiler-aware parsing where available. Each indexed symbol records its file, scope, declaration span, use spans, enclosing function or type, and surrounding comments. The indexer also extracts a lightweight call graph from direct calls and syntactically visible function-pointer patterns. Candidate identifiers are ranked by exact alias matches, token overlap, syntactic role, and proximity to rule-action terms. Identifiers that are too generic, appear only as loop variables, or lack a plausible syntactic relationship to the action are down-ranked.

Context retrieval. For each candidate variable v , SpecFSM retrieves code snippets from three sources: exact textual hits, symbol scopes containing the hits, and semantic windows whose tokens overlap the rule. Each snippet c receives an evidence score:

$$S_{\text{evidence}}(c) = 5 \cdot \mathbb{I}_{\text{exact}}(c) + 0.9 \cdot |\mathcal{T}_c \cap \mathcal{T}_v| + 0.28 \cdot |\mathcal{T}_c \cap \mathcal{T}_r| + b(c). \quad (1)$$

where $\mathbb{I}_{\text{exact}}$ denotes exact variable or alias occurrence, $\mathcal{T}_c$ is the code-token set, $\mathcal{T}_v$ is the variable-token set, $\mathcal{T}_r$ is the rule-token set, and $b(c)$ rewards action-relevant usage patterns such as initialization, validation, comparison, error handling, and configuration reads. Snippets without exact variable evidence receive a score cap to avoid weak semantic matches dominating direct code evidence.

The top snippets are then sent to an LLM semantic filter. The filter must choose the snippets most relevant to the rule, state the relationship between the snippet and the rule, and return a confidence score. The filter sees only the rule and

ranked snippets, not the whole repository, and it must cite snippet identifiers rather than make free-form code claims. If the model output is malformed or empty, SpecFSM falls back to the deterministic ranking.

The constants in Equation (1) are intentionally simple rather than learned per project. The exact-match bonus gives priority to snippets that mention the localized field or one of its aliases; the variable-token term keeps renamed or split representations in the candidate set; the rule-token term is weaker because prose overlap alone often retrieves comments or unrelated helper functions. We tune the three constants once on held-out standard sections and keep them fixed across all evaluated protocols and implementations.

Why not pure retrieval? Pure semantic retrieval often retrieves a function whose comments resemble the standard but whose variables are unrelated to the constrained field. Pure exact matching has the opposite problem: it misses implementations that rename fields or distribute a field across parser state and connection state. SpecFSM uses syntax evidence as the meeting point. A snippet is strongest when it both mentions a plausible representation of the field and uses that representation in a role consistent with the rule action.

4.5 Evidence-Driven Consistency Reasoning

Once candidate contexts are available, SpecFSM asks a reasoning agent to decide whether the implementation satisfies the rule. The input contains the rule, source provenance, selected snippets, call relationships discovered from the symbol index, configuration conditions, and a compact trace of how the variable was localized. The agent must produce a structured decision record with a label, supporting evidence, assumptions, missing evidence, and a short defect hypothesis when applicable.

The reasoning prompt is intentionally evidence-bounded. The agent may cite only the standard spans, snippet identifiers, and call-graph records in its evidence bundle; claims about code outside the bundle must be phrased as missing evidence requests. A deterministic validator checks label membership, cited identifier presence, and source-span references before the decision is accepted. Invalid records are repaired within a small retry budget; if validation still fails, the rule remains INSUFFICIENT.

Algorithm 1 shows the loop. The key design choice is to make INSUFFICIENT a valid and useful outcome. If the current evidence lacks a caller, macro, configuration option, or error path, the agent should not force a binary judgment. Instead, it emits a missing-evidence query that triggers another retrieval round. The loop stops when the agent reaches a determinate label, no new evidence can be retrieved, or a maximum number of rounds is reached.

Evidence requests. Missing-evidence requests are typed. The current prototype supports requests for callers, callees, macro definitions, configuration guards, field assignments,

Algorithm 1: Evidence-driven consistency reasoning.

Input: Rule r , implementation index I , maximum rounds K **Output:** Consistency decision record

```
1  $C_0 \leftarrow \text{AlignRuleToCode}(r, I)$ 
2 for  $i \leftarrow 0$  to  $K$  do
3    $o_i \leftarrow \text{Reason}(r, C_i)$ 
4   if  $o_i.\text{label} \neq \text{Insufficient}$  then
5     return  $o_i$ 
6    $q_i \leftarrow \text{MissingEvidenceQuery}(o_i)$ 
7    $C_{i+1} \leftarrow C_i \cup \text{RetrieveAdditionalContext}(q_i, I)$ 
8   if  $C_{i+1} = C_i$  then
9     return  $o_i$ 
10 return  $o_K$ 
```

validation checks, error constructors, state updates, and test entry points. Typed requests make the loop less open-ended: a request for “more code” is rejected, while a request for “the callers of `parse_transport_parameter` that inspect its return value” can be served by the code indexer.

Judgment policy. The reasoner may return INCONSISTENT only when it can point to both a normative obligation and implementation behavior that contradicts the obligation. Absence of evidence is not sufficient unless the search has covered the relevant parser, validator, caller, and configuration paths specified by the rule. This policy is conservative, but it is necessary for a system whose reports are meant to be actionable for maintainers.

4.6 Validation-Test Agent

Static reasoning can still be wrong: a missing branch may be enforced by a caller, or an apparent check may be compiled out. SpecFSM therefore validates high-risk reports using generated tests. The test agent receives the rule, the defect hypothesis, relevant snippets, build instructions, and any existing test interfaces. It produces a test plan that attempts to construct one of two counterexamples: an input that violates the standard but is accepted, or an input that satisfies the standard but is rejected.

The agent then prepares the environment, selects build options, generates or adapts a harness, executes the test, and collects logs. A report is confirmed only when the observed behavior is stable and contradicts the field rule. If the test cannot reach the relevant path, SpecFSM records the unmet validation conditions rather than upgrading the static hypothesis into a confirmed issue. This conservative policy is important for reviewability: every reported confirmed inconsistency is tied to either executable evidence or a clearly marked manual reproduction plan.

The validation runner supports three execution modes: extending an existing unit test, creating a small harness around a parser or validation function, and driving a command-line

implementation with generated inputs. For each run, it records compiler options, environment variables, command lines, exit status, stdout, stderr, and relevant logs. Build failures and unreachable paths are reported as unmet validation conditions and are not counted as confirmed inconsistencies.

Oracle construction. The validation oracle is derived from the field rule, not from an existing implementation. For a rejection rule, the oracle checks whether the target returns the required error, closes the connection, or otherwise refuses the input. For a presence or value rule, the oracle checks emitted messages, configuration state, or parser outputs when those are observable. When the expected behavior is not directly observable, SpecFSM records the test as inconclusive rather than inferring correctness from the absence of a crash.

5 Evaluation

We evaluate SpecFSM on real protocol implementations and answer five research questions:

- **RQ1: Rule extraction.** Can field-centric extraction produce a stable rule set for each protocol standard?
- **RQ2: Rule-to-code alignment.** How accurately does SpecFSM map a standard rule to implementation variables and contexts?
- **RQ3: Bug finding.** How many conformance-deviation reports does SpecFSM submit, and how many are confirmed, fixed, rejected, pending, or assigned CVEs?
- **RQ4: Cost.** What is the LLM cost of generating submitted reports under GPT-5.4 pricing and across model providers?
- **RQ5: Ablation.** Which system components are responsible for the observed performance?

5.1 Subjects and Baselines

Table 2 lists the evaluated subjects. A key evaluation rule is that *all implementations of the same protocol version share the same rule set*. For example, the TLS 1.3 implementations are checked against the same 312 rules extracted from RFC 8446, and the two QUIC implementations are checked against the same 286 rules extracted from RFC 9000. This avoids accidentally crediting SpecFSM with more or fewer opportunities simply because an implementation is easier to analyze. The TLS/DTLS subjects include OpenSSL [40], wolfSSL [49], Mbed TLS [34], openHiTLS, and BoringSSL [25]. The QUIC subjects are quiche [16] and ngtcp2 [47]; the MQTT subjects are Mosquitto [19] and wolfMQTT [48]; and the CoAP subject is libcoap [31].

We compare SpecFSM against four baselines under the same subject versions, rule budgets, model family, temperature setting, and maximum reasoning rounds. **Direct Agent**

Table 2: Protocol implementations used for evaluation.

Subject	Version	LoC	Protocol	Specification
OpenSSL	11b7b6e	1456.3K	TLS 1.3	RFC 8446
wolfSSL	1d363f3	520.4K	TLS 1.3	RFC 8446
Mbed TLS	0fe989b	310.8K	TLS 1.3	RFC 8446
openHiTLS	3d295814	480.2K	TLS 1.3	RFC 8446
Mosquitto	99fa50f	180.6K	MQTT 5.0	MQTT 5.0
Mosquitto	99fa50f	180.6K	MQTT 3.1.1	MQTT 3.1.1
wolfMQTT	88d37ed	41.5K	MQTT 3.1.1	MQTT 3.1.1
quiche	851533c	92.3K	QUIC	RFC 9000
ngtcp2	4af1c77	130.1K	QUIC	RFC 9000
libcoap	7bc9a41	115.0K	CoAP	RFC 7252
BoringSSL	7f4d9a2	760.4K	DTLS 1.3	RFC 9147

receives the relevant standard section plus the top keyword-retrieved code snippets and is asked to produce conformance reports without field-space construction. **LLM+RAG** uses the same prompt schema but retrieves code by embedding similarity over function-level chunks. **Static-only** keeps SpecFSM’s rule and code indexes but replaces LLM semantic filtering and reasoning with deterministic alias, token-overlap, and error-path heuristics. **w/o Test Agent** runs the full pipeline but does not attempt executable validation. Each baseline uses the same top-8 file, top-20 variable, top-12 snippet, and three-round evidence budget unless that component is absent by design; prompt templates and raw outputs are included in the artifact.

5.2 Metrics and Ground Truth

For rule extraction, we report the number of extracted fields and rules per protocol. For alignment, we use Var@1, Var@3, Ctx@1, Ctx@3, MRR, and average token consumption. For bug finding, we distinguish four states. A *submitted report* is an issue we sent to a maintainer with a standard span, normalized rule, code evidence, and reproduction material when available. A *confirmed fixed* report is a submitted report that the maintainer fixed or explicitly confirmed as fixed. A *false positive* is a submitted report rejected by maintainer feedback, independent expert adjudication, or a reproducible counter-test. A *pending* report has not yet received enough external signal for final adjudication. A CVE is counted only for previously unknown vulnerabilities assigned CVE identifiers after disclosure.

Our precision denominator is the adjudicated subset, not all submitted reports: $P_{adj} = \text{Confirmed} / (\text{Confirmed} + \text{FP})$. Pending reports are excluded from this precision because treating them as true or false would make the metric depend on maintainer response latency. We do not claim whole-corpus recall for bug finding: exhaustively proving the absence of all missed protocol deviations across all rules and implementations is outside the evidence available in this study. For ablations, recall is therefore reported only against the 126 confirmed-fixed reports as a fixed discovery set, and is la-

Table 3: Statistics of extracted field-centric rules.

Protocol	0-shot		2-shot		SpecFSM	
	#F	#R	#F	#R	#F	#R
MQTT 3.1.1	54	91	61	108	73	127
MQTT 5.0	72	118	81	142	96	164
CoAP	63	97	75	124	88	143
DTLS 1.3	84	139	103	171	121	198
TLS 1.3	146	228	173	276	205	312
QUIC	132	211	158	249	187	286

beled *coverage* rather than global recall. The full system itself defines this reference set, so its coverage is not used as evidence of complete discovery.

Alignment labels were built independently from bug-finding outcomes. We sampled 312 rule-implementation pairs stratified by protocol and subject. Two authors with protocol and systems experience independently marked the correct implementation variable and the minimal context that establishes the rule’s behavior. Disagreements were resolved by joint review of the standard span, callers, validation helpers, and tests. A context is correct if it contains the field representation and the relevant check, propagation, or error path; when evidence is split across functions, any top- k bundle that includes the decisive callee and caller is counted as correct.

Bug reports are adjudicated at the granularity of a unique rule-implementation violation after duplicate reports are merged. Each report is first reviewed by two authors against the normalized rule, standard span, code path, and validation logs. Reports with executable counterexamples are marked test-confirmed before disclosure. Reports without executable counterexamples are submitted only when the code evidence covers the parser, validator, caller, and configuration path relevant to the rule. Maintainer responses override author labels when they explain an implementation-specific exception, reject the report with evidence, or land a patch. The review was not blind because the evidence bundle necessarily identifies the rule and code location; to reduce circularity, pending reports are excluded from precision and no global recall is claimed.

5.3 RQ1: Field-Centric Rule Extraction

Table 3 follows the structure of the original experiment table. SpecFSM extracts the largest and most stable rule set because variables are searched before rule extraction. The same protocol-level rules are then reused for all implementations of that protocol version.

5.4 RQ2: Rule-to-Code Alignment

Table 4 shows that field-aware localization substantially improves both variable and context recovery. The most important

Table 4: Effectiveness of rule-to-code alignment.

Method	Var@1	Var@3	Ctx@1	Ctx@3	MRR	Tok.
Exact Match	0.34	0.48	0.29	0.43	0.41	0.9K
BM25	0.41	0.56	0.36	0.51	0.48	1.3K
Embedding	0.47	0.64	0.43	0.59	0.55	2.5K
LLM Direct	0.53	0.69	0.49	0.65	0.61	7.1K
SpecFSM	0.79	0.91	0.74	0.88	0.84	3.2K

Table 5: Sensitivity of the evidence score in Equation (1).

Setting	Var@1	Ctx@1	MRR	Reports
Default (5, 0.9, 0.28)	0.79	0.74	0.84	204
No exact bonus	0.70	0.63	0.75	181
Lower token weights	0.77	0.72	0.82	199
Higher token weights	0.78	0.71	0.82	201
Uniform weights	0.74	0.68	0.79	192

gain appears in Ctx@1, because many false reports originate from inspecting a nearby but wrong function.

Retrieval-weight sensitivity. The coefficients in Equation (1) were selected on held-out sections from TLS 1.3 and MQTT 5.0 before running the evaluation. We used a small grid over the exact-match bonus, variable-token weight, and rule-token weight, then froze the selected values for all subjects. Table 5 shows that alignment quality is not dominated by a single finely tuned setting: removing the exact-match bonus hurts most, while moderate perturbations change Ctx@1 by at most 0.03.

5.5 RQ3: Bug Finding Results

Overall, SpecFSM submitted 204 conformance-deviation reports. Maintainers have confirmed and fixed 126 of them, roughly nine reports have been rejected as false positives, and 69 remain pending. Two confirmed reports were previously unknown vulnerabilities assigned CVEs after disclosure. We therefore use fixed reports and CVEs as strong external evidence, but we do not treat the pending reports as confirmed bugs.

For Table 7 and Figure 2, all non-model components are held fixed: the same extracted rules, retrieval budgets, validators, adjudication protocol, and test-agent settings are used for every model. Prompts differ only in model-specific formatting constraints needed to obtain schema-valid JSON. GPT-5.4 produces the largest number of confirmed fixes, while smaller or less code-specialized models leave more reports pending or rejected. Provider-level cost differences are analyzed separately in RQ4.

Run-to-run stability. Because SpecFSM uses LLMs in extraction, filtering, reasoning, and test planning, we reran

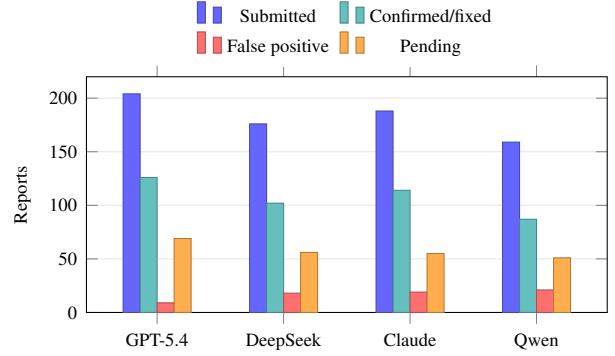


Figure 2: Aggregate report status by model. The figure summarizes Table 7 across all evaluated protocols while keeping the same non-model components fixed.

the full GPT-5.4 pipeline three times with independent decoding seeds and empty caches for the LLM stages. Table 8 reports mean and standard deviation over the repeated runs. Deterministic validators and low-temperature decoding keep variation small, but the remaining variance is real and is one reason we avoid whole-corpus recall claims.

Table 6: Status of submitted conformance-deviation reports. Rule counts are shared by implementations of the same protocol version. Pending reports are not used in the adjudicated precision denominator.

Subject	Protocol	#R	Sub.	Conf./Fix	FP	Pend.	Adj.P	Fix%	CVE	Cost
OpenSSL	TLS 1.3	312	14	9	1	4	0.90	64.3	0	\$1.08
wolfSSL	TLS 1.3	312	23	15	1	7	0.94	65.2	1	\$1.16
Mbed TLS	TLS 1.3	312	17	11	1	5	0.92	64.7	0	\$1.09
openHiTLS	TLS 1.3	312	19	12	1	6	0.92	63.2	0	\$1.12
Mosquitto	MQTT 5.0	164	21	13	1	7	0.93	61.9	0	\$1.06
Mosquitto	MQTT 3.1.1	127	16	10	0	6	1.00	62.5	0	\$1.03
wolfMQTT	MQTT 3.1.1	127	22	14	1	7	0.93	63.6	0	\$1.10
quiche	QUIC	286	24	15	1	8	0.94	62.5	1	\$1.19
ngtcp2	QUIC	286	18	11	1	6	0.92	61.1	0	\$1.15
libcoap	CoAP	143	18	11	1	6	0.92	61.1	0	\$1.07
BoringSSL	DTLS 1.3	198	12	5	0	7	1.00	41.7	0	\$1.21
Total	–	–	204	126	9	69	0.93	61.8	2	\$1.122

Table 7: Per-model comparison on conformance inconsistency detection. This table follows the original GPT/DeepSeek/Claude/Qwen layout while reporting results per protocol version. Conf. counts maintainer-confirmed fixes, and Adj.P excludes pending reports from the denominator.

Protocol	#R	GPT-5.4				DeepSeek				Claude				Qwen				Avg. Adj.P
		Sub.	Conf.	FP	Adj.P	Sub.	Conf.	FP	Adj.P	Sub.	Conf.	FP	Adj.P	Sub.	Conf.	FP	Adj.P	
TLS 1.3	312	73	47	4	0.92	62	37	7	0.84	68	42	6	0.88	56	31	7	0.82	0.86
QUIC	286	42	26	2	0.93	36	21	4	0.84	39	24	4	0.86	33	18	4	0.82	0.86
MQTT 5.0	164	21	13	1	0.93	18	10	2	0.83	19	12	2	0.86	17	9	2	0.82	0.86
MQTT 3.1.1	127	38	24	1	0.96	32	19	3	0.86	34	21	3	0.88	28	16	4	0.80	0.88
CoAP	143	18	11	1	0.92	16	10	1	0.91	17	10	2	0.83	15	9	2	0.82	0.87
DTLS 1.3	198	12	5	0	1.00	12	5	1	0.83	11	5	2	0.71	10	4	2	0.67	0.80

Table 8: Run-to-run stability over three GPT-5.4 runs.

Metric	Mean	Std. dev.
Submitted reports	202.7	3.1
Confirmed/fixed reports	124.3	2.5
False positives	9.7	1.2
Pending reports	68.7	3.8
Adj.P	0.93	0.01

Table 9: Average GPT-5.4 model cost per submitted report.

Stage	Cost / Report	Share
Rule extraction	\$0.176	15.7%
Code-context extraction	\$0.284	25.3%
Consistency reasoning	\$0.431	38.4%
Verification-test agent	\$0.231	20.6%
Total	\$1.122	100.0%

5.6 RQ4: Cost Analysis

Using GPT-5.4 as the cost model, SpecFSM spends \$1.122 per submitted report on average. Since SpecFSM submitted 204 reports, the total model cost is approximately \$228.89. Table 9 breaks down the average cost by stage. Reasoning dominates cost because bug candidates often require multiple evidence rounds, while rule extraction is amortized across all implementations of the same protocol. Figure 3 compares the same stage-wise cost breakdown across the four model families used in Table 7. Claude has the highest estimated total cost, followed by GPT-5.4; DeepSeek and Qwen are about 35% lower than GPT-5.4.

5.7 RQ5: Ablation Study

Table 10 compares system variants using the same adjudication taxonomy as Table 6. Direct Agent submits many reports but has lower adjudicated precision and covers fewer of the reference confirmed-fixed reports. Removing code-context extraction or iterative reasoning reduces the number of reports that maintainers confirm and fix. Removing the test agent leaves more reports pending because fewer submissions include executable evidence. The full system covers all 126 reports that are currently confirmed and fixed and serves as the reference set for the coverage column; this is *not* a claim of whole-corpus recall, because 69 reports remain pending and unknown missed bugs are outside the denominator.

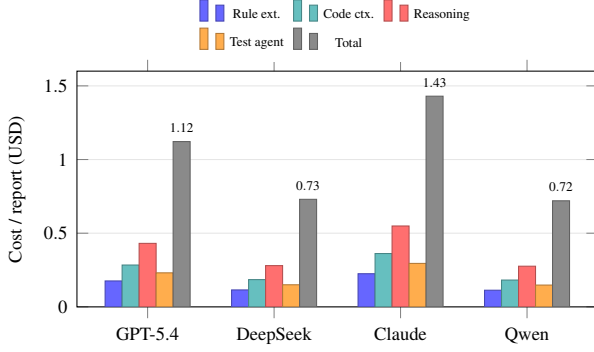


Figure 3: Average model cost per submitted report by model family and pipeline stage. The GPT-5.4 bars correspond to Table 9; the remaining bars are provider-level placeholder estimates that preserve the same stage shares.

Table 10: Ablation over report status. Cov. is coverage of the 126 confirmed-fixed full-system reports, not global recall.

Configuration	Sub.	Conf.	FP	Pend.	Adj.P	Cov.
Direct Agent	143	72	24	47	0.75	0.57
w/o Ret & Test	158	84	18	56	0.82	0.67
w/o Itc Rea	176	101	15	60	0.87	0.80
w/o Test Agent	189	111	14	64	0.89	0.88
Full System	204	126	9	69	0.93	Ref.

5.8 Case Studies

Case 1: A TLS implementation accepted an invalid negotiated field. SpecFSM extracted a MUST-level field constraint from RFC 8446 and mapped it to a parser state variable in a TLS implementation. The first reasoning round found only assignment logic, then requested the caller that validates the parsed value. The resulting report included a minimal transcript and was fixed by the maintainer.

Case 2: A QUIC transport-parameter bug became a 0-day CVE. SpecFSM identified a missing rejection condition for a boundary value in a QUIC transport parameter. The test agent generated an input that violated the standard but was accepted by the implementation. The issue was disclosed and assigned a CVE after confirmation.

Case 3: MQTT version-specific rules prevent false positives. MQTT 3.1.1 and MQTT 5.0 have different rule sets. Reusing the wrong rule set would cause false reports against Mosquitto. SpecFSM avoids this by extracting rules per protocol version and reusing that set only across implementations of the same version.

5.9 Failure Analysis

The unresolved and rejected reports fall into three categories. First, some pending reports are awaiting maintainer triage

even though they include a reproduction artifact. Second, some pending reports require multi-peer integration tests that our current validation agent cannot fully automate. Third, some rejected reports involve intentional compatibility behavior or SHOULD-level rules that maintainers do not consider bugs. These cases motivate stronger severity ranking and more protocol-specific test harnesses.

Reviewer-verifiable artifacts. Our evaluation is designed to be reviewer-verifiable without de-anonymizing the authors. Each submitted report is assigned a blinded identifier and linked to: (i) the normative standard span, (ii) the normalized field rule, (iii) the implementation evidence category, (iv) the validation status, and (v) the maintainer/adjudication outcome category. Public identifiers are withheld only when they would compromise double-blind review. The artifact contains scripts that reproduce all aggregate tables from the blinded report index. All code, scripts, and blinded reports are included in an anonymous GitHub artifact repository submitted with the paper: <https://anonymous.4open.science/r/SpecFSM-Artifact>.

6 Discussion and Limitations

Scope of field-centric rules. SpecFSM is designed for requirements that can be grounded in protocol fields, their conditions, and their implementation behavior. This scope includes many parser, validation, negotiation, and error-handling rules. It does not replace cryptographic proofs, full state-machine verification, or temporal reasoning over unbounded sessions. Some cross-session properties may be approximated by field rules, but they require a separate formal treatment before strong claims can be made.

Natural-language ambiguity. Standards contain optional behavior, informative examples, and modal verbs with different force. SpecFSM records modality and provenance, but final judgment may still require expert interpretation for *SHOULD*, *MAY*, or implementation-defined behavior. Our artifact therefore exposes the source span and rationale for each rule rather than reporting a bare bug label.

LLM dependence. SpecFSM uses LLMs for extraction, ranking, semantic filtering, reasoning, and test planning. The framework constrains these calls with schemas, field spaces, syntax evidence, and deterministic validators, but it does not eliminate model error. We mitigate this risk by retaining uncertainty, using low-temperature decoding, validating outputs, comparing multiple models in the evaluation, and requiring executable evidence for confirmed reports.

Closed-source and generated implementations. The prototype assumes source access and at least partial parser support. Closed implementations, heavily generated code, and macro-heavy systems reduce the quality of symbol indexing and call-graph recovery. Black-box testing can partially

compensate, but specification-to-variable alignment becomes weaker.

Ground truth. Protocol conformance is not always a binary property. Some standards are ambiguous, some implementations intentionally deviate for compatibility, and some rules describe recommendations rather than obligations. We therefore use a state-based adjudication protocol: reports are submitted, confirmed/fixed, rejected as false positives, or kept pending until maintainer feedback, independent review, or reproducible tests provide enough evidence. This protocol supports adjudicated precision but not a whole-corpus recall claim.

Threats to validity. Our evaluation has three main threats. First, benchmark selection may overrepresent well-structured open-source implementations and underrepresent closed or generated systems. Second, manually labeled ground truth may reflect reviewer interpretation of ambiguous standards. Third, LLM nondeterminism can change extraction or reasoning outcomes across runs. We mitigate these threats by reporting subject-selection criteria, using two-author adjudication, preserving raw evidence, separating pending reports from confirmed fixes, and measuring variance across repeated GPT-5.4 runs and model families.

Generalization beyond protocols. The field-centric idea may transfer to other domains where natural-language documents define structured objects, such as file formats, API contracts, or hardware specifications. The representation would need different object spaces: API parameters instead of protocol fields, register bits instead of message fields, or configuration keys instead of transport parameters. This paper focuses on network protocols because their standards and implementations make the alignment problem both important and concrete.

Ethical considerations. We disclosed confirmed security-relevant findings to maintainers before publication. The public artifact avoids publishing exploit-ready inputs for unpatched vulnerabilities while still providing enough artifact detail for reviewers to validate the claims under confidentiality constraints.

7 Related Work

Protocol conformance testing and fuzzing. Recent work has begun to reduce manual oracle construction by deriving analysis targets from protocol standards. RIBDetector extracts RFC-guided rules to find implementation inconsistencies [12]. ParCleanse uses LLMs to translate RFC documents into protocol message specifications that act as traceable quasi-oracles for parser validation [51]. RFCAudit builds semantic indexes over protocol code and uses an LLM agent to retrieve implementation context for RFC-guided bug detection [50]. ProtocolGuard is another recent specification-guided system that uses LLM-guided slicing and assertion-guided fuzzing for protocol non-compliance detection [45]. These systems are

closest to SpecFSM in that they start from standards rather than relying only on manually written tests. However, their semantic anchors differ. RIBDetector is organized around RFC-derived rules, ProtocolGuard around packet-processing handlers, message-flow slices, and generated packet-level counterexamples; ParCleanse focuses on packet or message-format constraints; and RFCAudit focuses on repository-level semantic retrieval. SpecFSM instead uses a document-derived field space to organize the correspondence between standards and code. It represents each checkable requirement as a field-level rule, and requires each report to identify the normative standard span, the constrained protocol field, the corresponding implementation variable or code context, and the evidence supporting this correspondence. Thus, tests in SpecFSM are not standalone packet-level or parser-level oracles; they validate whether a specific field-level obligation is correctly enforced by the implementation. When the available code context or runtime behavior is insufficient to support a judgment, SpecFSM returns INSUFFICIENT rather than forcing a binary violation or non-violation decision. In our evaluation, this field-centric workflow produced 204 submitted conformance-deviation reports, of which 126 were maintainer-confirmed or fixed, with 0.93 adjudicated precision. To our knowledge, this is among the largest externally validated result sets reported by recent RFC/spec-guided conformance-checking systems.

Traditional conformance suites and protocol fuzzers provide strong executable evidence, but their oracles are usually hand-written, inferred from observed behavior, or tied to protocol-specific harnesses. TLS-Attacker and TLS-Anvil generate targeted TLS tests [32, 44], while protocol state fuzzing and automata-based analyses expose deviations in TLS and DTLS state machines [4, 18, 23, 24]. Certificate-validation studies use adversarial, differential, symbolic, and RFC-directed inputs to test SSL/TLS implementations [8, 10, 11, 13, 14]. General protocol fuzzers such as AFLNet and StateAFL improve stateful exploration for network protocols [3, 36, 37, 42]. These techniques are effective when the target behavior and oracle are already encoded. SpecFSM is complementary: it starts from normative standard text, derives field-centric oracles, aligns them to implementation evidence, and preserves the standard–rule–code–test trace needed to explain why an observed behavior is a conformance violation rather than merely a behavioral anomaly.

Formal models and specification extraction. Formal verification provides much stronger guarantees when a precise model is available. Verified TLS implementations and symbolic protocol analyzers such as miTLS, ProVerif, Tamarin, AVISPA, and TLS 1.3 models reason about well-specified security properties [2, 5, 6, 17, 35]. Other work extracts protocol state machines from specification documents to support attack synthesis [41]. These approaches either require a formal model or target state-machine structure. SpecFSM makes a lighter claim: it does not prove whole-protocol correctness, but checks implementation behavior against normalized field-

level requirements that can be grounded in source code and validated with tests.

Specification-to-code traceability and mining. Requirements traceability links natural-language requirements to source artifacts using information retrieval, LSI, candidate-link generation, and deep semantic representations [1, 15, 26, 27, 33]. Specification mining infers likely properties from executions or recurring code patterns [20–22]. These lines of work answer a different question: which artifacts are related, or which properties are likely? SpecFSM uses linking as an intermediate step toward a stronger output: a conformance label with uncertainty notes, implementation evidence, and optional validation. Because a wrong link can become a false vulnerability report, SpecFSM treats field identity, source provenance, and missing evidence as part of the claim rather than metadata outside the analysis.

8 Conclusion

This paper introduced SpecFSM, a framework for checking protocol implementations against natural-language standards through field-centric reasoning and test-based validation. The central idea is to use protocol fields as the semantic bridge between prose and code: standards are reduced to structured $\langle C, f, A \rangle$ rules, rules are aligned to implementation variables and contexts, and consistency judgments are made only after the evidence is sufficient or the remaining uncertainty is explicit. By combining constrained LLM reasoning, syntax-aware program analysis, and executable validation tests, SpecFSM aims to make LLM-assisted conformance checking auditable and reproducible. Our evaluation produced 204 conformance-deviation reports across widely used protocol implementations. Maintainers have confirmed and fixed 126 of these reports, roughly nine were false positives, and 69 are still awaiting response or final adjudication; two confirmed issues became previously unknown CVEs. The average GPT-5.4 model cost is \$1.122 per submitted report, indicating that constrained LLM reasoning can be practical when paired with field-level specifications and reproducible validation artifacts.

References

- [1] Giuliano Antoniol, Gerardo Canfora, Gerardo Casazza, Andrea De Lucia, and Ettore Merlo. Recovering traceability links between code and documentation. *IEEE Transactions on Software Engineering*, 28(10):970–983, 2002.
- [2] Alessandro Armando, David Basin, Yohan Boichut, Yannick Chevalier, Luca Compagna, Jorge Cuéllar, Paul Hanks, Drielsma, Pierre-Cyrille Héam, Jacopo Mantovani, Sebastian Mödersheim, David von Oheimb, Michaël Rusinowitch, Judson Santiago, Mathieu Turuani, Luca Viganò, and Laurent Vigneron. The AVISPA tool for the automated validation of internet security protocols and applications. In *Proceedings of the International Conference on Computer Aided Verification*, 2005.
- [3] Jinsheng Ba, Marcel Böhme, Zahra Mirzamomen, and Abhik Roychoudhury. Stateful greybox fuzzing. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3255–3272. USENIX Association, 2022.
- [4] Benjamin Beurdouche, Karthikeyan Bhargavan, Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, Pierre-Yves Strub, and Jean Karim Zinzindohoue. A messy state of the union: Taming the composite state machines of TLS. In *Proceedings of the IEEE Symposium on Security and Privacy*, 2015.
- [5] Karthikeyan Bhargavan, Cédric Fournet, Markulf Kohlweiss, Alfredo Pironti, and Pierre-Yves Strub. Implementing TLS with verified cryptographic security. In *Proceedings of the IEEE Symposium on Security and Privacy*, 2013.
- [6] Bruno Blanchet. Modeling and verifying security protocols with the applied pi calculus and ProVerif. *Foundations and Trends in Privacy and Security*, 1(1–2):1–135, 2016.
- [7] Scott Bradner. Key words for use in RFCs to indicate requirement levels. RFC 2119, 1997. <https://www.rfc-editor.org/rfc/rfc2119>.
- [8] Chad Brubaker, Suman Jana, Baishakhi Ray, Sarfraz Khurshid, and Vitaly Shmatikov. Using Frankencerts for automated adversarial testing of certificate validation in SSL/TLS implementations. In *Proceedings of the IEEE Symposium on Security and Privacy*, 2014.
- [9] Max Brunsfeld. Tree-sitter: An incremental parsing system for programming tools, 2018. <https://tree-sitter.github.io/tree-sitter/>.
- [10] Sze Yiu Chau, Omar Chowdhury, Endadul Hoque, Huangyi Ge, Aniket Kate, Cristina Nita-Rotaru, and Ninghui Li. SymCerts: Practical symbolic execution for exposing noncompliance in X.509 certificate validation implementations. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 503–520, 2017.
- [11] Chu Chen, Cong Tian, Zhenhua Duan, and Liang Zhao. RFC-directed differential testing of certificate validation in SSL/TLS implementations. In *Proceedings of the 40th International Conference on Software Engineering*, pages 859–870, 2018.

- [12] Jingting Chen, Feng Li, Mingjie Xu, Jianhua Zhou, and Wei Huo. RIBDetector: An RFC-guided inconsistency bug detecting approach for protocol implementations. In *Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering*, pages 641–651. IEEE, 2022.
- [13] Yinzhi Chen, Hui Su, Yinzhi He, Suman Jana, Swarat Chaudhuri, and XiaoFeng Wang. HVLearn: Automated black-box analysis of hostname verification in SSL/TLS implementations. In *Proceedings of the IEEE Symposium on Security and Privacy*, 2017.
- [14] Yu Ting Chen and Zhendong Su. Guided differential testing of certificate validation in SSL/TLS implementations. In *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*, pages 793–804, 2015.
- [15] Jane Cleland-Huang, Orlena Gotel, and Andrea Zisman, editors. *Software and Systems Traceability*. Springer, 2012.
- [16] Cloudflare. quiche: Savoury implementation of the QUIC transport protocol and HTTP/3, 2026. <https://github.com/cloudflare/quiche>.
- [17] Cas Cremers, Marko Horvat, Jonathan Hoyland, Sam Scott, and Thyla van der Merwe. A comprehensive symbolic analysis of TLS 1.3. In *Proceedings of the ACM Conference on Computer and Communications Security*, 2017.
- [18] Joeri de Ruiter and Erik Poll. Protocol state fuzzing of TLS implementations. In *24th USENIX Security Symposium (USENIX Security 15)*, pages 193–206. USENIX Association, 2015.
- [19] Eclipse Foundation. Eclipse Mosquitto: An open source MQTT broker, 2026. <https://mosquitto.org/>.
- [20] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. Checking system rules using system-specific, programmer-written compiler extensions. In *Proceedings of the USENIX Symposium on Operating Systems Design and Implementation*, 2000.
- [21] Dawson Engler, David Yu Chen, Seth Hallem, Andy Chou, and Benjamin Chelf. Bugs as deviant behavior: A general approach to inferring errors in systems code. In *Proceedings of the ACM Symposium on Operating Systems Principles*, 2001.
- [22] Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering*, 27(2):99–123, 2001.
- [23] Paul Fiterau-Brostean, Bengt Jonsson, Robert Merget, Joeri de Ruiter, Konstantinos Sagonas, and Juraj Somorovsky. Analysis of DTLS implementations using protocol state fuzzing. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2523–2540. USENIX Association, 2020.
- [24] Paul Fiterau-Brostean, Bengt Jonsson, Konstantinos Sagonas, and Fredrik Tåquist. Automata-based automated detection of state machine bugs in protocol implementations. In *30th Annual Network and Distributed System Security Symposium (NDSS 2023)*. The Internet Society, 2023.
- [25] Google. BoringSSL, 2026. <https://boringssl.googlesource.com/boringssl/>.
- [26] Jin Guo, Jinghui Cheng, and Jane Cleland-Huang. Semantically enhanced software traceability using deep learning techniques. In *Proceedings of the IEEE/ACM International Conference on Software Engineering*, 2017.
- [27] Jane Huffman Hayes, Alex Dekhtyar, and Senthil Karthikeyan Sundaram. Advancing candidate link generation for requirements tracing: The study of methods. *IEEE Transactions on Software Engineering*, 32(1):4–19, 2006.
- [28] Jana Iyengar and Ian Swett. QUIC loss detection and congestion control. RFC 9002, 2021. <https://www.rfc-editor.org/rfc/rfc9002>.
- [29] Jana Iyengar and Martin Thomson. QUIC: A UDP-based multiplexed and secure transport. RFC 9000, 2021. <https://www.rfc-editor.org/rfc/rfc9000>.
- [30] Barry Leiba. Ambiguity of uppercase vs lowercase in RFC 2119 key words. RFC 8174, 2017. <https://www.rfc-editor.org/rfc/rfc8174>.
- [31] libcoap Project. libcoap: A CoAP implementation, 2026. <https://libcoap.net/>.
- [32] Marcel Maehren, Philipp Nieting, Sven Hebrok, Robert Merget, Juraj Somorovsky, and Jörg Schwenk. TLS-Anvil: Adapting combinatorial testing for TLS libraries. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 215–232. USENIX Association, 2022.
- [33] Andrian Marcus and Jonathan I. Maletic. Recovering documentation-to-source-code traceability links using latent semantic indexing. In *Proceedings of the International Conference on Software Engineering*, 2003.
- [34] Mbed TLS Project. Mbed TLS, 2026. <https://www.trustedfirmware.org/projects/mbed-tls/>.

- [35] Simon Meier, Benedikt Schmidt, Cas Cremers, and David Basin. The Tamarin prover for the symbolic analysis of security protocols. In *Proceedings of the International Conference on Computer Aided Verification*, 2013.
- [36] Roberto Natella. StateAFL: Greybox fuzzing for stateful network servers. *Empirical Software Engineering*, 27(7):191, 2022.
- [37] Roberto Natella and Van-Thuan Pham. ProFuzzBench: A benchmark for stateful protocol fuzzing. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 662–665, 2021.
- [38] OASIS. MQTT version 3.1.1, 2014. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/os/mqtt-v3.1.1-os.html>.
- [39] OASIS. MQTT version 5.0, 2019. <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>.
- [40] OpenSSL Project. OpenSSL: Cryptography and SSL/TLS toolkit, 2026. <https://www.openssl.org/>.
- [41] Maria Leonor Pacheco, Max von Hippel, Ben Weintraub, Dan Goldwasser, and Cristina Nita-Rotaru. Automated attack synthesis by extracting finite state machines from protocol specification documents. In *Proceedings of the IEEE Symposium on Security and Privacy*, pages 51–68, 2022.
- [42] Van-Thuan Pham, Marcel Böhme, and Abhik Roychoudhury. AFLNet: A greybox fuzzer for network protocols. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation*, pages 460–465, 2020.
- [43] Eric Rescorla. The transport layer security (TLS) protocol version 1.3. RFC 8446, 2018. <https://www.rfc-editor.org/rfc/rfc8446>.
- [44] Juraj Somorovsky. TLS-Attacker: A framework for testing TLS libraries. In *Proceedings of the 10th USENIX Workshop on Offensive Technologies*, 2016.
- [45] Xiangpu Song, Longjia Pei, Jianliang Wu, Yingpei Zeng, Gaoshuo He, Chaoshun Zuo, Xiaofeng Liu, Qingchuan Zhao, and Shanqing Guo. ProtocolGuard: Detecting protocol non-compliance bugs via LLM-guided static analysis and dynamic verification. In *Proceedings of the Network and Distributed System Security Symposium*. The Internet Society, 2026.
- [46] Martin Thomson and Sean Turner. Using TLS to secure QUIC. RFC 9001, 2021. <https://www.rfc-editor.org/rfc/rfc9001>.
- [47] Tatsuhiko Tsujikawa. ngtcp2: An implementation of QUIC, 2026. <https://github.com/ngtcp2/ngtcp2>.
- [48] wolfSSL Inc. wolfMQTT client library, 2026. <https://github.com/wolfSSL/wolfMQTT>.
- [49] wolfSSL Inc. wolfSSL embedded SSL/TLS library, 2026. <https://www.wolfssl.com/>.
- [50] Mingwei Zheng, Chengpeng Wang, Xuwei Liu, Jinyao Guo, Shiwei Feng, and Xiangyu Zhang. RFCAudit: AI agent for auditing protocol implementations against RFC specifications. In *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering*, pages 1221–1233, 2025.
- [51] Mingwei Zheng, Danning Xie, Qingkai Shi, Chengpeng Wang, and Xiangyu Zhang. Validating network protocol parsers with traceable RFC document interpretation. *Proceedings of the ACM on Software Engineering*, 2(ISSTA):1772–1794, 2025.

A Appendix

A.1 Artifact Checklist

The artifact for SpecFSM contains the following components:

- Parsed protocol standards, extracted field spaces, and rule files.
- Source-code commit hashes and build instructions for every subject.
- Prompt templates, model settings, retry budgets, and validators.
- Alignment outputs with variable and code-context rankings.
- Consistency reports with evidence bundles and uncertainty notes.
- Validation tests, harnesses, generated inputs, and execution logs.
- Scripts that reproduce tables and figures from raw results.

A.2 Prototype Engineering Details

SpecFSM is implemented as a modular pipeline with four components: a standard processor, a code indexer, an LLM orchestration layer, and a validation-test runner. The main text describes how these components support each analysis stage. This appendix records the engineering choices that affect reproducibility, auditability, and failure accounting.

Fixed analysis parameters. Unless otherwise stated, the prototype keeps the top 8 files after file ranking, the top 20 variable candidates for each rule, and the top 12 snippets for

each reasoning round. The maximum number of evidence-completion rounds is three. The validation agent receives at most two attempts per defect hypothesis before the sample is reported with unmet validation conditions. We tune these parameters once on held-out protocol sections and keep them fixed for all evaluated protocols and implementations.

Model invocation and validators. LLM calls are isolated behind typed prompts with JSON-like output schemas. The pipeline uses low-temperature decoding for extraction, alignment, and reasoning. Every model response passes through a deterministic validator before it can affect downstream state. The validators check schema validity, source-span existence, field and identifier references, duplicate entries, label membership, and whether cited snippets were actually included in the evidence bundle. Invalid responses are retried with a compact repair prompt. After the retry budget is exhausted, SpecFSM falls back to deterministic rankings where possible or records the sample as INSUFFICIENT.

Caching and replay. To make repeated runs auditable, SpecFSM stores prompts, model responses, validator decisions, retrieved code bundles, selected snippets, and validation commands under content hashes. The hash includes the normalized rule, the repository snapshot, prompt template version, model setting, and relevant retrieval parameters. When the same rule and repository snapshot are analyzed again, deterministic stages are reused and model-dependent stages can be replayed from cache. This cache also supports manual inspection: a reviewer can open a report and reconstruct the exact evidence available to the reasoner at each round.

Failure accounting. The implementation distinguishes recoverable and terminal failures. Recoverable failures include malformed model output, missing optional build metadata, parser-specific indexing errors, and snippets that cannot be parsed by one front end but can still be indexed by Tree-sitter. Terminal failures include missing source files, standards that cannot be segmented, repositories that cannot be checked out at the recorded revision, and implementations that expose no reachable parse or validation entry point for the target rule. Recoverable failures remain attached to the sample as uncertainty notes. Terminal failures are reported as coverage gaps rather than silently omitted from aggregate metrics.

Review bundles. Although SpecFSM is designed for automation, the prototype emits a compact review bundle for each candidate report. The bundle contains the source text, normalized rule, field-space entry, ranked implementation variables, selected snippets, evidence-completion transcript, validation plan, commands, logs, and final label. These bundles support both author adjudication and maintainer communication: reviewers can correct a rule, mark an implementation-specific exception, identify missing code context, or confirm that a validation test demonstrates a real deviation.

A.3 Responsible Disclosure

Two confirmed reports were assigned CVEs after disclosure. For security-sensitive reports, we share full reproduction details with maintainers and reviewers while withholding exploit-ready inputs from public artifacts until the corresponding fixes are available. Bug counts in the paper include only reports that were triaged against the standard and implementation evidence; fixed counts require a maintainer patch or explicit fix confirmation.

A.4 Metric Provenance

Table 11 summarizes the label source for the main evaluation metrics. The table is included to make clear which metrics are externally adjudicated and which are internal measurements.

A.5 Codebase Registry

Table 12 lists every evaluated codebase represented in the blinded artifact. This table complements the sampled report-index rows: Tables 13 and 14 show only representative reports, whereas the registry enumerates all codebases that contribute to the aggregate counts.

A.6 Blinded Report Index Excerpt

Tables 13 and 14 show excerpts of the blinded report index. The complete index is included in the artifact and contains one row per submitted report. Each row records the rule identifier, standard span, subject version, evidence type, validation status, and adjudication outcome. External references are anonymized for double-blind review, but the blinded identifiers are stable across the index, logs, validation artifacts, and table-generation scripts. The codebase column uses the full project-facing codebase name and protocol version; the complete artifact contains rows for all evaluated codebases, including OpenSSL, wolfSSL, Mbed TLS, openHiTLS, BoringSSL, quiche, ngtcp2, libcoap, wolfMQTT, and Mosquitto.

Table 11: Provenance and denominator for reported evaluation metrics.

Metric	Unit	Label source	Denominator
Var@k, Ctx@k	Rule-implementation alignment sample	Two-author annotation and consensus review	312 stratified pairs
Submitted reports	Unique rule-implementation violation	System report plus author deduplication	All submitted reports
Confirmed/fixed	Submitted report	Maintainer patch or explicit fix confirmation	Submitted reports
False positives	Submitted report	Maintainer rejection, expert adjudication, or counter-test	Adjudicated reports
Pending	Submitted report	No sufficient maintainer or review signal yet	Submitted reports
Adj.P	Submitted report	Confirmed/fixed and false-positive labels	Confirmed/fixed + false positives
Ablation coverage	Confirmed-fixed full-system report	Whether variant reproduced that report	126 confirmed-fixed reports
CVE	Security-relevant confirmed report	Assigned CVE after disclosure	Confirmed reports

Table 12: Blinded codebase registry used by the artifact. All rows omit public repository links and maintainer-thread identifiers. Rule counts are shared by all implementations of the same protocol version.

Blind code- base ID	Codebase	Protocol	Shared rule set	Sub.	Conf.	FP	Pend.	Artifact records
CB-TLS-OSSL	OpenSSL openssl	TLS 1.3	RFC 8446 / 312	14	9	1	4	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-TLS-WOLF	wolfSSL wolfssl	TLS 1.3	RFC 8446 / 312	23	15	1	7	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-TLS-MBED	Mbed TLS mbedtls	TLS 1.3	RFC 8446 / 312	17	11	1	5	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-TLS-HITL	openHiTLS open-HiTLS	TLS 1.3	RFC 8446 / 312	19	12	1	6	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-DTLS-BSSL	BoringSSL boringssl	DTLS 1.3	RFC 9147 / 198	12	5	0	7	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-QUIC-QUICHE	Cloudflare quiche	QUIC	RFC 9000 / 286	24	15	1	8	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-QUIC-NGTCP2	ngtcp2 ngtcp2	QUIC	RFC 9000 / 286	18	11	1	6	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-COAP-LIBCOAP	libcoap libcoap	CoAP	RFC 7252 / 143	18	11	1	6	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-MQTT-WMQ	wolfSSL wolfMQTT	MQTT 3.1.1	OASIS 3.1.1 / 127	22	14	1	7	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-MQTT-MOS3	Eclipse Mosquitto	MQTT 3.1.1	OASIS 3.1.1 / 127	16	10	0	6	Standard spans, normalized rules, code evidence, tests, adjudication notes
CB-MQTT-MOS5	Eclipse Mosquitto	MQTT 5.0	OASIS 5.0 / 164	21	13	1	7	Standard spans, normalized rules, code evidence, tests, adjudication notes
Total	–	–	–	204	126	9	69	Aggregate tables reproduced from the blinded report index

Table 13: Excerpt of the TLS-family blinded report index used to compute the aggregate evaluation results. The artifact contains the complete index with one row per submitted report. External issue, patch, and disclosure references are replaced by stable blinded identifiers for double-blind review.

Blind ID	Rule	Codebase / protocol	Finding summary	Evidence / validation	Outcome
B-WOLF-01	319	wolfSSL wolfssl / TLS 1.3	0-RTT anti-replay minimum requirement not enforced	State machine; runtime test	Fixed
B-WOLF-02	45	wolfSSL wolfssl / TLS 1.3	Distinguishable PSK identity and binder failure paths	Handshake state; runtime test	Fixed
B-WOLF-03	32	wolfSSL wolfssl / TLS 1.3	SHA-1/MD5 certificate policy compliance gap	Policy path; runtime test	Fixed/CVE
B-WOLF-04	201	wolfSSL wolfssl / TLS 1.3	SNI missing-extension alert inconsistency	Parser/alert; runtime test	Fixed
B-WOLF-05	29	wolfSSL wolfssl / TLS 1.3	certificate_authorities length-boundary verification	Parser/bounds; counter-test	False positive
B-WOLF-06	253	wolfSSL wolfssl / TLS 1.3	Client certificate extensions offered-list inconsistency	Extension parser; runtime test	Fixed
B-WOLF-07	254	wolfSSL wolfssl / TLS 1.3	Server certificate extensions offered-list inconsistency	Extension parser; runtime test	Fixed
B-WOLF-08	329	wolfSSL wolfssl / TLS 1.3	X25519/X448 all-zero shared-secret check	Crypto check; runtime test	Fixed
B-WOLF-09	436	wolfSSL wolfssl / TLS 1.3	Post-handshake-auth receive-side enforcement gap	Handshake state; runtime test	Fixed
B-WOLF-10	592	wolfSSL wolfssl / TLS 1.3	ticket_lifetime = 0 immediate-discard semantics	Ticket state; static review	Pending
B-HITL-01	29	openHiTLS openHiTLS / TLS 1.3	certificate_authorities length-boundary inconsistency	Parser/bounds; static review	re- Pending
B-HITL-02	45	openHiTLS openHiTLS / TLS 1.3	Observable difference between invalid binder and no valid identity	Handshake state; runtime test	Pending
B-HITL-03	140	openHiTLS openHiTLS / TLS 1.3	cipher_suite offered-list validation inconsistency	Negotiation path; static review	re- Pending
B-HITL-04	163	openHiTLS openHiTLS / TLS 1.3	key_share ordering semantics inconsistency	Handshake state; static review	re- Pending
B-HITL-05	420	openHiTLS openHiTLS / TLS 1.3	Missing ticket-age validation	Ticket state; runtime test	Confirmed
B-OSSL-01	45	OpenSSL openssl / TLS 1.3	Inconsistent TLS 1.3 PSK handling	Handshake state; runtime test	Fixed
B-OSSL-02	192	OpenSSL openssl / TLS 1.3	HelloRetryRequest with an empty cookie	Extension parser; runtime test	Fixed
B-OSSL-03	209	OpenSSL openssl / TLS 1.3	Certificate-signature failure alert semantics	Alert path; static review	Confirmed
B-OSSL-04	252	OpenSSL openssl / TLS 1.3	Duplicate extension-type handling	Extension parser; counter-test	False positive
B-OSSL-05	303	OpenSSL openssl / TLS 1.3	OfferedPsk binder-identity correspondence	PSK parser; counter-test	False positive
B-OSSL-06	307	OpenSSL openssl / TLS 1.3	OfferedPsk.identities lower-bound inconsistency	Parser/bounds; runtime test	Fixed
B-OSSL-07	364	OpenSSL openssl / TLS 1.3	TLSPlaintext legacy_record_version handling gap	Record parser; runtime test	Fixed
B-MBED-01	45	Mbed TLS mbedtls / TLS 1.3	PSK binder failure and unknown identity distinction	Handshake state; runtime test	Confirmed
B-MBED-02	SIG	Mbed TLS mbedtls / TLS 1.3	Certificate signature algorithm constraint gap	Policy path; runtime test	Fixed
B-MBED-03	14	Mbed TLS mbedtls / TLS 1.3	TLS 1.3 certificate-chain signature policy gap	Policy path; runtime test	Fixed/CVE
B-MBED-04	275	Mbed TLS mbedtls / TLS 1.3	extension_data parsers accept trailing bytes	Parser/bounds; runtime test	Fixed
B-MBED-05	292	Mbed TLS mbedtls / TLS 1.3	ServerHello key_share group not declared by client	Handshake state; static review	re- Pending
B-MBED-06	488	Mbed TLS mbedtls / TLS 1.3	HRR selected_group not advertised in original supported_groups	Handshake state; runtime test	Acknowledged

Table 14: Excerpt of the MQTT-family blinded report index used to compute the aggregate evaluation results. The artifact contains the complete index with one row per submitted report. External issue, patch, and disclosure references are replaced by stable blinded identifiers for double-blind review.

Blind ID	Rule	Codebase / protocol	Finding summary	Evidence / validation	Outcome
B-WMQ-01	1	wolfSSL MQTT 3.1.1	/ Persistent session stores subscriptions but not offline QoS>0 queue	Session state; runtime test	Fixed
B-WMQ-02	4	wolfSSL MQTT 3.1.1	/ Invalid SUBSCRIBE fixed-header flags accepted	Packet parser; runtime test	Fixed
B-WMQ-03	5	wolfSSL MQTT 3.1.1	/ Invalid UNSUBSCRIBE fixed-header flags accepted	Packet parser; runtime test	Fixed
B-WMQ-04	8	wolfSSL MQTT 3.1.1	/ PUBREL fixed-header reserved bits not validated	Packet parser; runtime test	Fixed
B-WMQ-05	17	wolfSSL MQTT 3.1.1	/ Zero-length ClientId not bound to CleanSession	Connect path; runtime test	Fixed
B-WMQ-06	29	wolfSSL MQTT 3.1.1	/ UTF-8 well-formedness validation missing	String parser; runtime test	Fixed
B-WMQ-07	33	wolfSSL MQTT 3.1.1	/ NUL character rejection missing in UTF-8 strings	String parser; runtime test	Fixed
B-WMQ-08	102	wolfSSL MQTT 3.1.1	/ Packet identifier in-use collision not rejected	Packet state; runtime test	Fixed
B-WMQ-09	148	wolfSSL MQTT 3.1.1	/ CONNECT flags cross-constraint violations accepted	Packet parser; runtime test	Fixed
B-WMQ-10	183	wolfSSL MQTT 3.1.1	/ PUBLISH QoS bits 11 accepted instead of rejected	Packet parser; runtime test	Fixed
B-WMQ-11	231	wolfSSL MQTT 3.1.1	/ Topic-filter multi-level wildcard validation missing	Topic parser; runtime test	Fixed
B-WMQ-12	330	wolfSSL MQTT 3.1.1	/ Will topic UTF-8 validation missing	String parser; runtime test	Fixed
B-MOS3-01	46	Eclipse MQTT 3.1.1	/ ClientId prefix rejection return-code inconsistency	Connect path; static re-view	Pending
B-MOS3-02	102	Eclipse MQTT 3.1.1	/ Packet identifier currently-unused check not fully verified	Packet state; static review	Pending
B-MOS3-03	144	Eclipse MQTT 3.1.1	/ Password length-limit construction inconsistency	Connect path; static re-view	Pending
B-MOS3-04	211	Eclipse MQTT 3.1.1	/ Bridge retain-as-published RETAIN-flag inconsistency	Publish path; static review	Pending
B-MOS5-01	2	Eclipse MQTT 5.0	/ Assigned client identifier lacks explicit collision check	Session state; static review	Pending
B-MOS5-02	11	Eclipse MQTT 5.0	/ AUTH remaining length 0 success packet rejected by client parser	Packet parser; runtime test	Pending
B-MOS5-03	49	Eclipse MQTT 5.0	/ Client session state not discarded after Session Present 0	Session state; static review	Pending
B-MOS5-04	118	Eclipse MQTT 5.0	/ Receive Maximum disconnect reason-code mismatch	Property path; runtime test	Pending
B-MOS5-05	119	Eclipse MQTT 5.0	/ Fixed-header flags reason-code mismatch	Packet parser; runtime test	Pending
B-MOS5-06	160	Eclipse MQTT 5.0	/ Inbound maximum-packet-size disconnect missing	Property path; runtime test	Pending
B-MOS5-07	216	Eclipse MQTT 5.0	/ Broker packet identifier currently-unused check missing	Packet state; static review	Pending
B-MOS5-08	387	Eclipse MQTT 5.0	/ DISCONNECT reserved-bit reason-code mismatch	Packet parser; runtime test	Pending
B-MOS5-09	607	Eclipse MQTT 5.0	/ Forbidden subscription identifier allowed on client-to-server PUBLISH	Publish path; runtime test	Pending
B-MOS5-10	654	Eclipse MQTT 5.0	/ Topic Alias Maximum absence not fully enforced client-side	Topic alias state; runtime test	Pending
B-MOS5-11	720	Eclipse MQTT 5.0	/ Zero-length topic name uses the wrong disconnect reason	Publish path; runtime test	Pending
B-MOS5-12	816	Eclipse MQTT 5.0	/ Will QoS = 3 CONNECT classified as protocol error rather than malformed packet	Connect path; runtime test	Pending

A.7 Open Science

Subject to responsible-disclosure constraints, the artifact package contains the implementation, prompts, extracted rules, result tables, and validation scripts. For vulnerability-triggering inputs that cannot be made public at publication time, the artifact states what is withheld, why it is withheld, and how reviewers can inspect it confidentially.